

MÓDULO 2 · CURSO IAC UCM

Código Terraform reutilizable

AGENDA

1 - Calidad y convenciones

1. Calidad y convenciones

2. Tipos, validación y expresiones

3. Iteración

4. Módulos

terraform fmt : formato canónico

Un único estilo oficial: sangría de 2 espacios, `=` alineados, espacios normalizados.

`fmt` solo toca espacios y alineación, nunca el significado. En VS Code: extensión HashiCorp Terraform y, en **Ajustes**, activar **Format On Save**. Al guardar un `.tf`, el editor ejecuta `fmt`.

Antes

```
resource "aws_instance" "web" {  
ami = data.aws_ami.ubuntu.id  
    instance_type="t3.micro"  
}
```

Después de fmt

```
resource "aws_instance" "web" {  
    ami          = data.aws_ami.ubuntu.id  
    instance_type = "t3.micro"  
}
```

Variantes de `fmt`

Variante	Qué hace	Cuándo
<code>terraform fmt</code>	Formatea los <code>.tf</code> del directorio actual	Mientras se escribe
<code>terraform fmt -recursive</code>	Entra también en subcarpetas (módulos)	Antes de cada commit
<code>terraform fmt -check -recursive</code>	No modifica; falla si algo no está formateado	En CI

terraform validate: coherencia antes del plan

Revisa la configuración **sin llamar a la API de AWS**. Necesita `init`: el esquema viene del provider.

Sintaxis

HCL bien formado.

Argumentos

Existen en el esquema del recurso y tienen el tipo correcto.

Referencias

Variables, recursos y data sources referenciados están declarados.

terraform validate: ejemplo

main.tf

```
resource "aws_instance" "web" {  
  ami          = data.aws_ami.ubuntu.id  
  instance_typ = "t3.micro"  
}
```

terraform validate

```
Error: Unsupported argument  
on main.tf line 3:  
3:   instance_typ = "t3.micro"  
An argument named "instance_typ" is not  
expected here. Did you mean "instance_type"?
```

Qué no detecta `validate`

Lo que solo sabe AWS

- Nombre de bucket ya ocupado
- Tipo de instancia inexistente en la región
- Permiso IAM que falta
- Aparecen en `plan` o en `apply`

No confundir con `validation {}`

- `terraform validate`: comando, revisa la **estructura** del código
- `validation {}`: dentro de una variable, revisa el **valor** recibido
- Se ve en el bloque 2

Rutina antes de cada commit

1

fmt -recursive

Errores de formato



2

validate

Estructura, sin AWS



3

plan

Efecto real. Leerlo entero

validate

- No autentica, no lee el estado y no llama a la API
- No hace falta perfil ni permisos IAM
- Basta el esquema del provider, ya descargado con `init`

plan

- También revisa la estructura, pero no arranca sin acceso
- Necesita credenciales con permiso de lectura
- Consulta AWS y el estado para calcular el cambio real

`validate` no sustituye al `plan`: es el filtro previo. Caza la estructura en local o en CI, sin permisos de AWS. El `plan` responde a otra pregunta: qué cambiaría en la cuenta.

Ficheros de un módulo raíz

Terraform lee todos los `.tf`: el nombre del fichero es solo organización.

Fichero	Contenido
<code>versions.tf</code>	Bloque <code>terraform {}</code> : <code>required_version</code> y <code>required_providers</code>
<code>provider.tf</code>	Bloques <code>provider</code>
<code>variables.tf</code>	Variables de entrada, en orden alfabético
<code>locals.tf</code>	Locals usados en varios ficheros
<code>main.tf</code>	Resources, data y llamadas a módulos
<code>outputs.tf</code>	Salidas, en orden alfabético
<code>modules/</code>	Módulos hijo locales: <code>./modules/<nombre></code>

versions.tf y el lock file

versions.tf o terraform.tf

HashiCorp lo llama `terraform.tf`; Google y la comunidad, `versions.tf`. Ambos valen: lo importante es que todo el equipo use el mismo.

.terraform.lock.hcl va a Git

Registra versión exacta y hashes de cada provider. Equipo y pipeline usan los mismos binarios. En el laboratorio 1 se ignoró para simplificar.

Nombres: `snake_case`

Minúsculas y palabras separadas por `_`. Sin mayúsculas, guiones ni espacios.

Estilo	Ejemplo	¿Válido para Terraform?
<code>snake_case</code>	<code>servidor_web</code>	Sí, es la convención
<code>kebab-case</code>	<code>servidor-web</code>	Lo acepta, pero no es la convención
<code>camelCase</code>	<code>servidorWeb</code>	Lo acepta, pero no es la convención

Se aplica a nombres locales de recursos y data sources, variables, outputs, locals y módulos.

Nombre en Terraform frente a nombre en AWS

Nombre de Terraform

- Siempre `snake_case`
- Solo existe en tu código y en el estado

```
resource "aws_s3_bucket" "datos_crudos" {  
  bucket = "ucm-ana-datos-crudos"  
}
```

Nombre en AWS

- Reglas de cada servicio
- Un bucket S3 no admite `_`
- Laboratorio 1: `replace()` pasó `microcredencial_ucm` a `microcredencial-ucm`

No repetir el tipo en el nombre local

La dirección completa ya incluye el tipo.

Correcto	Redundante
<code>aws_security_group.web</code>	<code>aws_security_group.web_security_group</code>
<code>aws_subnet.publica_a</code>	<code>aws_subnet.subnet_publica_a</code>
<code>aws_iam_role.lambda</code>	<code>aws_iam_role.lambda_role</code>

Orden de los argumentos en un bloque

Cada grupo separado del siguiente por una línea en blanco.



Una instancia EC2 ordenada

```
resource "aws_instance" "servidor_web" {
  ami           = data.aws_ami.ubuntu.id
  instance_type = var.tipo_instancia
  subnet_id     = aws_subnet.publica_a.id

  tags = {
    Name = "servidor-web"
  }

  root_block_device {          # disco del sistema: 20 GB cifrado
    volume_size = 20
    encrypted   = true
  }

  lifecycle {
    prevent_destroy = true  # Terraform rechaza un plan que la destruya
  }

  depends_on = [aws_internet_gateway.principal]
}
```

Terraform acepta cualquier orden. Este pone arriba lo importante (qué se repite, qué es) y las excepciones al final.

Etiquetas comunes con `default_tags`

```
provider "aws" {  
  region = var.region  
  
  default_tags {  
    tags = {  
      Proyecto    = "microcredencial-ucm"  
      Alumno      = var.identificador_alumno  
      Gestionado  = "terraform"  
    }  
  }  
}
```

Sustituye al `merge(local.etiquetas_comunes, ...)` de la clase 1. Los `tags` del recurso se suman; si coincide la clave, gana el recurso. `Alumno` permite filtrar consola y factura.

2 - Tipos, validación y expresiones

1. Calidad y convenciones

2. Tipos, validación y expresiones

3. Iteración

4. Módulos

Por qué hacen falta tipos

Primitivos

`string`, `number`, `bool`. Un solo valor: una región, un número de instancias.

Colección

`list`, `set`, `map`. Varios valores del mismo tipo: zonas, tipo por entorno.

Estructurales

`object`. Atributos con nombre y tipo propio: la configuración de una subred.

Declarar `type`: el `plan` rechaza valores con otra forma y quien lee sabe qué pasar. Con `type = any` se pierden ambas cosas.

list: secuencia ordenada

Mismo tipo, **orden concreto**, índice desde 0, admite repetidos.

```
variable "zonas_disponibilidad" {
  description = "Zonas de disponibilidad, por orden de preferencia"
  type        = list(string)
  default     = ["eu-west-1a", "eu-west-1b", "eu-west-1c"]
}

locals {
  zona_principal = var.zonas_disponibilidad[0]      # "eu-west-1a"
  num_zonas     = length(var.zonas_disponibilidad) # 3
}
```

Una **AZ** es un centro de datos independiente dentro de la región. **eu-west-1** tiene tres: repartir recursos entre ellas da alta disponibilidad.

set : sin orden ni repetidos

Un elemento está o no está; no tiene posición.

```
variable "cidrs_administracion" {  
    description = "Redes desde las que se permite administrar la infraestructura"  
    type        = set(string)  
    default     = ["10.0.0.0/16", "192.168.1.0/24"]  
}
```

Se usa cuando solo importa la pertenencia: redes autorizadas, usuarios, nombres de zonas.

Mismos datos en terraform console

Ambas variables declaradas con `["eu-west-1b", "eu-west-1a", "eu-west-1b"]`.

list(string)

```
> var.zonas_lista
tolist([
  "eu-west-1b",
  "eu-west-1a",
  "eu-west-1b",
])
> var.zonas_lista[0]
"eu-west-1b"
```

set(string)

```
> var.zonas_set
toset([
  "eu-west-1a",
  "eu-west-1b",
])
> var.zonas_set[0]
Error: Invalid index
```

list frente a set

	list	set
Orden	El de la declaración	No tiene; se muestra ordenado alfabéticamente
Repetidos	Se mantienen	Se eliminan
Acceso	Por índice: <code>var.x[0]</code>	Se recorre entero o <code>contains()</code>
Identidad	Su posición	Su propio valor
Conversión	<code>tolist(set)</code>	<code>toset(lista)</code>

La identidad decide el bloque 3: si se borra un elemento intermedio, en una lista cambian las posiciones; en un set los valores siguen igual.

map : claves con nombre

```
variable "tipo_instancia_por_entorno" {  
  description = "Tamaño de las instancias EC2 en cada entorno"  
  type        = map(string)  
  default = {  
    dev  = "t3.micro"  
    pre  = "t3.small"  
    prod = "t3.large"  
  }  
}  
  
resource "aws_instance" "app" {  
  ami          = data.aws_ami.ubuntu.id  
  instance_type = var.tipo_instancia_por_entorno[var.entorno]  
}
```

El **tipo de instancia** fija CPU, memoria y precio. Clave inexistente: error. `lookup(mapa, "qa", "t3.micro")` da un valor por defecto.

object : estructura con atributos fijos

```
variable "subred_publica" {
  description = "Configuración de la subred pública"
  type = object({
    cidr      = string
    zona      = string
    publica   = bool
  })
}

resource "aws_subnet" "publica" {
  vpc_id            = aws_vpc.principal.id
  cidr_block        = var.subred_publica.cidr
  availability_zone = var.subred_publica.zona
  map_public_ip_on_launch = var.subred_publica.publica
}
```

Una **subred** es un rango de IPs de la VPC ligado a una AZ. Es pública si sus instancias tienen IP pública y ruta a Internet.

map frente a object

Se parecen al escribirlos; no son lo mismo.

map(string)

- Claves libres: se pueden añadir o quitar
- Todos los valores del mismo tipo
- Datos de la misma naturaleza: etiquetas, tipo por entorno

object({...})

- Claves fijas: las define el tipo
- Cada atributo con su tipo
- Configuración de una cosa: una subred, una instancia

Atributos opcionales con `optional()`

```
variable "subredes" {  
  type = map(object({  
    cidr    = string  
    zona    = string  
    publica = optional(bool, false)    # si no se indica: false  
  }))  
  default = {  
    publica-a = { cidr = "10.0.1.0/24", zona = "eu-west-1a", publica = true }  
    app-a     = { cidr = "10.0.11.0/24", zona = "eu-west-1a" }  
  }  
}
```

Terraform ≥ 1.3. `optional(number)` sin segundo argumento toma `null`.

Bloque `validation`: el contenido

`type` comprueba la forma

- «Es un `string`»
- Falla si llega una lista o un número

`validation` comprueba el contenido

- «Es uno de los permitidos»
- Si la condición es falsa, el `plan` se detiene con tu mensaje, antes de llamar a AWS
- Usa funciones: `contains`, `can`, `regex`

Restringir tipos de instancia

Evita lanzar por error una máquina cara.

```
variable "tipo_instancia" {
  description = "Tipo de instancia EC2"
  type        = string
  default     = "t3.micro"

  validation {
    condition     = contains(["t3.micro", "t3.small", "t3.medium"], var.tipo_instancia)
    error_message = "Tipos permitidos en el laboratorio: t3.micro, t3.small, t3.medium."
  }
}
```

```
$ terraform plan -var tipo_instancia=m5.24xlarge
Error: Invalid value for variable
Tipos permitidos en el laboratorio: t3.micro, t3.small, t3.medium.
```

Funciones: colecciones

Catálogo fijo: Terraform no admite funciones definidas por el usuario.

Función	Ejemplo	Resultado
<code>length</code>	<code>length(var.zonas_disponibilidad)</code>	<code>3</code>
<code>contains</code>	<code>contains(["dev", "prod"], var.entorno)</code>	<code>true</code> / <code>false</code>
<code>toset</code> / <code>tolist</code>	<code>toset(["a", "b", "a"])</code>	set <code>["a", "b"]</code>
<code>keys</code> / <code>values</code>	<code>keys(var.subredes)</code>	lista de claves
<code>merge</code>	<code>merge(local.etiquetas, { Name = "web" })</code>	mapa combinado
<code>lookup</code>	<code>lookup(var.tipo_instancia_por_entorno, "qa", "t3.micro")</code>	valor o el por defecto

Funciones: texto, nulos y ficheros

Función	Ejemplo	Resultado
<code>format</code>	<code>format("worker-%02d", 3)</code>	<code>"worker-03"</code>
<code>replace</code>	<code>replace("ana_garcia", "_", "-")</code>	<code>"ana-garcia"</code>
<code>lower</code> / <code>upper</code>	<code>lower("ETL")</code>	<code>"etl"</code>
<code>coalesce</code>	<code>coalesce(var.nombre, "por-defecto")</code>	primer valor no nulo
<code>can</code> / <code>try</code>	<code>try(var.config.puerto, 80)</code>	valor o alternativa
<code>fileset</code>	<code>fileset("\${path.module}/datos", "*.csv")</code>	ficheros del patrón

`coalesce`

Primer valor que no es `null` ni `""`. Si leer el valor falla, el plan se detiene.

`try`

Primera expresión que no da error. Si el valor existe y es `null`, lo devuelve: no pasa al siguiente.

3 - Iteración

1. Calidad y convenciones
2. Tipos, validación y expresiones

3. Iteración

4. Módulos

Tres mecanismos para no repetir código

#

count

Multiplica un `resource`, `data` o `module`. Identifica cada copia por **posición**: `[0]`, `[1]`.

K

for_each

Multiplica un `resource`, `data` o `module`. Identifica cada copia por **clave**: `["app-a"]`.

{ }

dynamic

Repite un **bloque anidado** dentro de un recurso. No genera copias en el estado.

`count` y `for_each` son meta-argumentos: los entiende Terraform, no el provider. Un bloque no puede llevar los dos.

count : N copias

`count.index` vale 0, 1, 2... dentro del bloque.

```
resource "aws_instance" "worker" {
  count = var.num_workers  # 3

  ami          = data.aws_ami.ubuntu.id
  instance_type = "t3.micro"
  subnet_id    = aws_subnet.app.id

  tags = {
    Name = format("worker-%02d", count.index + 1)
  }
}
```

Plan: `aws_instance.worker[0]`, `[1]`, `[2]` (`worker-01 ... 03`). Subir a 5 añade `[3]` y `[4]`; bajar a 2 destruye `[2]`.

Recursos condicionales: condición ? 1 : 0

Un **NAT Gateway** da salida a Internet a las subredes privadas sin exponerlas. Cuesta ~0,05 \$/h + tráfico: sí en prod, no en dev.

```
resource "aws_eip" "nat" {
  count = var.crear_nat ? 1 : 0

  domain = "vpc"
}

resource "aws_nat_gateway" "principal" {
  count = var.crear_nat ? 1 : 0

  allocation_id = aws_eip.nat[0].id
  subnet_id     = aws_subnet.publica.id
}

output "nat_id" {
  value = one(aws_nat_gateway.principal[*].id) # null si no existe
}
```

Lo que muestra el plan

terraform plan

```
# aws_subnet.app[1] must be replaced
-/+ resource "aws_subnet" "app" {
  ~ cidr_block = "10.0.12.0/24"
    -> "10.0.13.0/24" # forces replacement
```

aws_subnet.app[2] will be destroyed

Plan: 1 to add, 0 to change, 2 to destroy.

Conclusiones

- Se pedía borrar una subred: destruye dos y crea una
- `10.0.13.0/24` se recrea en otra AZ y arrastra sus instancias
- Leer el plan entero antes de aplicar
- `count` solo para copias intercambiables o condicionales

`for_each` : una copia por elemento

Recibe un **mapa** o un **set de strings**. Una `list` no vale: hay que pasarla por `toSet()`.

Con un mapa

- `each.key` : la clave
- `each.value` : el valor (p. ej. un objeto)

Con un set

- `each.key` : el elemento
- `each.value` : el mismo elemento

for_each con un set: vuestros usuarios IAM

Así los creó el módulo 0:

```
resource "aws_iam_user" "alumno" {  
  for_each = toset(var.usuarios)  
  
  name = "ucm-${each.key}"  
  path = "/ucm/"  
}
```

Estado: `aws_iam_user.alumno["ana_garcia"]`, `["luis_perez"]` ... Dar de baja a un alumno solo destruye su usuario.

for_each con un mapa de objetos

```
resource "aws_subnet" "principal" {  
  for_each = var.subredes  
  
  vpc_id            = aws_vpc.principal.id  
  cidr_block        = each.value.cidr  
  availability_zone  = each.value.zona  
  map_public_ip_on_launch = each.value.publica  
  
  tags = {  
    Name = each.key  
  }  
}
```

Estado: `aws_subnet.principal["publica-a"]`, `["publica-b"]`, `["app-a"]`, `["app-b"]`. Quitar `app-b` solo destruye `["app-b"]`.

Restricciones de `for_each`

Claves conocidas en el plan

No sirven ids ni ARNs que AWS asigna en el `apply`. Error: cannot be determined until apply.

Sin listas

Una `list` no vale directamente: `toset()`.

Sin valores sensibles

Las claves no pueden ser `sensitive`.

Referenciar recursos con varias copias

`count` se comporta como una **lista**; `for_each`, como un **mapa**.

```
# count: splat [*] para sacar un atributo de todas las copias
output "ips_privadas_workers" {
  value = aws_instance.worker[*].private_ip
}

# for_each: se recorre con una expresión for
output "ids_subredes" {
  value = { for nombre, s in aws_subnet.principal : nombre => s.id }
}

# Una copia concreta
output "subred_app_a" {
  value = aws_subnet.principal["app-a"].id
}
```

¿count o for_each?

Situación	Elección
Crear o no un recurso según un <code>bool</code>	<code>count = var.x ? 1 : 0</code>
N copias idénticas e intercambiables	<code>count</code>
Elementos con identidad propia (subredes, usuarios, entornos)	<code>for_each</code>
Cada copia necesita argumentos distintos	<code>for_each</code> sobre un mapa de objetos
La colección puede cambiar de orden o perder elementos	<code>for_each</code>

Con moderación: dos recursos fijos se leen mejor como dos bloques explícitos que como un mapa de dos elementos.

dynamic : repetir un bloque anidado

Una **política IAM** dice qué acciones se permiten sobre qué recursos; cada **statement** es una regla.

```
variable "permisos_buckets" {
  type = map(list(string))
  default = {
    ucm-datasets    = ["s3:GetObject"]
    ucm-resultados = ["s3:GetObject", "s3:PutObject"]
  }
}

data "aws_iam_policy_document" "acceso_datos" {
  dynamic "statement" {
    for_each = var.permisos_buckets

    content {
      effect    = "Allow"
      actions   = statement.value
      resources = ["arn:aws:s3:::${statement.key}/*"]
    }
  }
}
```

JSON de la política

Un `statement` por bucket. El data source lo escribe en local y añade `Version`.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::ucm-datasets/*"
    },
    {
      "Effect": "Allow",
      "Action": ["s3:PutObject", "s3:GetObject"],
      "Resource": "arn:aws:s3:::ucm-resultados/*"
    }
  ]
}
```

Una sola acción queda como string. Varias, como lista ordenada.

Anatomía de `dynamic`

`dynamic` `"statement"`

Nombre del bloque anidado que se repite.

`for_each`

La colección. Admite listas, sets y mapas.

`content { }`

Cuerpo de cada bloque generado.

Iterador

Se llama como el bloque:
`statement.key /`
`.value`. Renombrable con
`iterator`.

`aws_iam_policy_document` genera el JSON en local, sin llamar a AWS: con los permisos del curso se ve en el `plan`.

Refactorizar sin destruir: moved

Cambiar `count` por `for_each`, renombrar o mover a un módulo cambia la **dirección en el estado**.

Sin moved

- Terraform ve un recurso que desaparece y otro nuevo
- Plan: destruir el viejo y crear el nuevo
- En las subredes, arrastra a sus instancias

Con moved (≥ 1.1)

- Mismo objeto, otra dirección
- Plan: `has moved to`
- `0 to add, 0 to change, 0 to destroy`

Bloques moved

```
# Antes: aws_subnet.app con count. Ahora: aws_subnet.principal con for_each.
moved {
  from = aws_subnet.app[0]
  to    = aws_subnet.principal["app-a"]
}

moved {
  from = aws_subnet.app[1]
  to    = aws_subnet.principal["app-b"]
}

# Recurso que pasa a vivir dentro de un módulo (bloque 4)
moved {
  from = aws_s3_bucket.raw
  to    = module.zona["raw"].aws_s3_bucket.this
}
```

Se quedan en el código mientras alguien pueda tener un estado con la dirección antigua.

4 - Módulos

1. Calidad y convenciones
2. Tipos, validación y expresiones

3. Iteración

4. Módulos

Qué es un módulo

Una carpeta con ficheros `.tf`. Todo proyecto Terraform ya es un módulo.

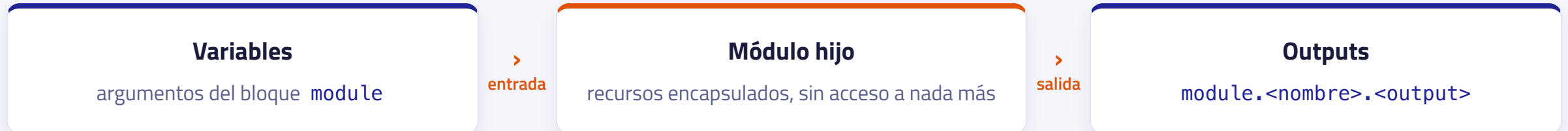
Módulo raíz

- Directorio donde se ejecuta `plan` / `apply`
- Tiene el estado y la configuración de providers

Módulo hijo

- Lo llama otro con un bloque `module`
- Local, del Registry o de un repositorio Git
- Admite `count` y `for_each`, como un recurso

Un módulo es una interfaz



Ámbito: qué ve cada módulo

El hijo no ve el raíz

Ni variables, ni locals, ni recursos. Solo lo que recibe como argumento.

El raíz no ve el hijo

Solo sus `output`.

Providers heredados

El hijo no declara `provider`

`required_providers` sí

El hijo indica qué providers y versiones mínimas necesita.

Cuándo crear un módulo

Un módulo tiene coste: otra interfaz que mantener y documentar.

Merece la pena

- Patrón repetido con parámetros distintos (bucket con cifrado, bloqueo público y ciclo de vida)
- Imponer un estándar: no se puede olvidar el cifrado
- Grupo con sentido como unidad (VPC + subredes + rutas)

No merece la pena

- Envolver un único recurso
- Exponer todos sus argumentos como variables
- Añade una capa sin quitar complejidad

Estructura estándar: `modules/bucket_datos/`

Fichero	Contenido
<code>versions.tf</code>	<code>required_providers</code> , sin bloque <code>provider</code>
<code>variables.tf</code>	Entradas con <code>type</code> , <code>description</code> y <code>validation</code>
<code>main.tf</code>	Recursos
<code>outputs.tf</code>	Lo que el raíz puede consumir
<code>README.md</code>	Propósito, entradas, salidas, ejemplo de uso

El README de entradas y salidas se puede generar con **terraform-docs**.

this y main

Con un solo recurso de ese tipo, el nombre es neutro. AWS acepta `this` o `main`. El curso usa `this`.

`this` · convención del curso

Cada tipo aparece una vez, así que el bucket y el versionado se llaman `this`. `aws_s3_bucket.this.id` es el id del bucket.

```
resource "aws_s3_bucket" "this" {  
  bucket = var.nombre  
}  
resource "aws_s3_bucket_versioning" "this" {  
  bucket = aws_s3_bucket.this.id  
}
```

- [AWS · nombres \(`main` o `this` \)](#)
- [Registry · terraform-aws-modules](#)

`main` · Google Cloud

La guía de estilo de Google nombra `main` a ese recurso único. AWS admite las dos formas.

Si hay dos del mismo tipo, el nombre indica el papel:
`aws_iam_role.lambda` y `aws_iam_role.glue`.

- [Google · estilo \(`main` \)](#)
- [Google · módulos reutilizables](#)

Taller guiado: módulo `bucket_datos`

El bucket del laboratorio 1 no tenía protección. El módulo impone los controles; quien lo usa decide nombre, ciclo de vida y etiquetas.



Bloqueo público

Las cuatro opciones activas.
No se desactiva desde fuera.



Cifrado SSE-S3

AES256 por defecto.
Tampoco configurable.



Versionado

S3 guarda versiones anteriores; expiran a los 30 días.



Ciclo de vida

Reglas opcionales: paso a `STANDARD_IA` y expiración.

versions.tf del módulo

```
terraform {  
  required_version = ">= 1.5.0"  
  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = ">= 5.0"  
    }  
  }  
}
```

Módulo: versión mínima `>=`

Si acotara por arriba, no se podría actualizar el provider sin tocar el módulo.

Raíz: versión concreta `~>`

Quien ejecuta fija la rama: `~> 5.0`.

variables.tf: nombre validado

```
variable "nombre" {
  description = "Nombre global del bucket. Debe empezar por ucm-."
  type       = string

  validation {
    condition     = can(regex("^ucm-[a-z0-9][a-z0-9-]{1,57}[a-z0-9]$", var.nombre))
    error_message = "Empieza por ucm-, solo minúsculas, números y guiones, máximo 63 caracteres."
  }
}

variable "force_destroy" {
  description = "Permite destruir el bucket aunque tenga objetos. Solo para laboratorios."
  type       = bool
  default    = false
}

variable "etiquetas" {
  description = "Etiquetas adicionales del bucket."
  type       = map(string)
  default    = {}
}
```

variables.tf : reglas de ciclo de vida

```
variable "reglas_ciclo_vida" {
  description = "Reglas de ciclo de vida. Lista vacía: sin configuración de ciclo de vida."
  type = list(object({
    id                = string
    prefijo            = optional(string, "")
    dias_transicion_ia = optional(number)
    dias_expiracion    = optional(number)
  }))
  default = []

  validation {
    condition = alltrue([
      for r in var.reglas_ciclo_vida : coalesce(r.dias_transicion_ia, 30) >= 30
    ])
    error_message = "S3 exige al menos 30 días antes de pasar a STANDARD_IA."
  }
}
```

main.tf: bucket y bloqueo público

```
resource "aws_s3_bucket" "this" {  
    bucket          = var.nombre  
    force_destroy   = var.force_destroy  
  
    tags = var.etiquetas  
}  
  
resource "aws_s3_bucket_public_access_block" "this" {  
    bucket = aws_s3_bucket.this.id  
  
    block_public_acls       = true  
    block_public_policy     = true  
    ignore_public_acls     = true  
    restrict_public_buckets = true  
}
```

No hay variables para desactivar el bloqueo: es el estándar que impone el módulo.

main.tf: cifrado y versionado

```
resource "aws_s3_bucket_server_side_encryption_configuration" "this" {
  bucket = aws_s3_bucket.this.id

  rule {
    apply_server_side_encryption_by_default {
      sse_algorithm = "AES256"
    }
  }
}

resource "aws_s3_bucket_versioning" "this" {
  bucket = aws_s3_bucket.this.id

  versioning_configuration {
    status = "Enabled"
  }
}
```

main.tf: ciclo de vida

```
resource "aws_s3_bucket_lifecycle_configuration" "this" {
  count = length(var.reglas_ciclo_vida) > 0 ? 1 : 0 # la API rechaza una configuración sin reglas

  bucket = aws_s3_bucket.this.id

  dynamic "rule" {
    for_each = var.reglas_ciclo_vida
    content {
      id      = rule.value.id
      status  = "Enabled"
      filter { prefix = rule.value.prefijo }

      dynamic "transition" {
        for_each = rule.value.dias_transicion_ia == null ? [] : [rule.value.dias_transicion_ia]
        content {
          days          = transition.value
          storage_class = "STANDARD_IA"
        }
      }
      # dynamic "expiration": mismo patrón con dias_expiracion

      noncurrent_version_expiration { noncurrent_days = 30 }
    }
  }

  depends_on = [aws_s3_bucket_versioning.this]
}
```

Patrones del bloque 3 en el módulo

count condicional

Sin reglas, no se crea la configuración de ciclo de vida.

dynamic "rule"

Repite un bloque por cada regla de la lista.

dynamic opcional

transition y **expiration** aparecen solo si se indican los días.

depends_on

Recomendación del provider: ciclo de vida con el versionado ya activo.

Con versionado, expirar solo marca el objeto como borrado; **noncurrent_version_expiration** limpia las versiones antiguas.

`outputs.tf` del módulo

```
output "arn" {
  description = "ARN del bucket"
  value       = aws_s3_bucket.this.arn
}

output "id" {
  description = "Nombre del bucket (en S3, id y nombre coinciden)"
  value       = aws_s3_bucket.this.id
}
```

Raíz: un data lake con tres zonas

Un **data lake** organiza los datos por grado de elaboración. Cada zona es un bucket con su retención.



Raíz: declarar las variables

El raíz decide qué buckets existen. `identificador_alumno` entra en el nombre; `zonas` dice cuántas llamadas habrá y qué ciclo de vida lleva cada una.

```
variable "identificador_alumno" {
  description = "Identificador del alumno, en minúsculas."
  type        = string
}

variable "zonas" {
  description = "Zonas del data lake y sus reglas de ciclo de vida."
  type = map(object({
    reglas_ciclo_vida = optional(list(object({
      id            = string
      prefijo       = optional(string, "")
      dias_transicion_ia = optional(number)
      dias_expiracion  = optional(number)
    })), [])
  }))
  default = {
    raw      = { reglas_ciclo_vida = [{ id = "expirar-crudos", dias_expiracion = 90 }] }
    curated  = { reglas_ciclo_vida = [{ id = "historico-a-ia", dias_transicion_ia = 30 }] }
    analytics = {}
  }
}
```

Raíz: Llamar a `bucket_datos`

Una entrada de `var.zonas` es una instancia del módulo. Los argumentos son las variables declaradas dentro de `modules/bucket_datos/`.

```
locals {
  alumno = replace(var.identificador_alumno, "_", "-")
}

module "zona" {
  source    = "../modules/bucket_datos"
  for_each = var.zonas

  nombre          = "ucm-${local.alumno}-${each.key}"
  reglas_ciclo_vida = each.value.reglas_ciclo_vida
  force_destroy   = true
  etiquetas       = { Zona = each.key }
}

output "buckets" {
  description = "Bucket de cada zona"
  value       = { for zona, m in module.zona : zona => m.id }
}
```

Flujo del taller



`state list` muestra `module.zona["raw"].aws_s3_bucket.this`: el módulo es una instancia más en el estado y `this` encaja con cualquier zona.

Avance de la clase 3

Estado remoto

S3 con bloqueo nativo:

```
use_lockfile =  
true (≥ 1.10).
```

DynamoDB,
obsoleto.

Operaciones de estado

```
state  
list/show/mv/rm,  
bloques import y  
removed.
```

Entornos

Workspaces frente a un directorio por entorno.

Pruebas y CI

```
terraform test,  
TFLint, Checkov o  
Trivy, pipeline con  
fmt -check.
```

Condiciones y secretos

```
precondition,  
check,  
sensitive,  
valores efímeros,  
write-only.
```

Takeaways

1. `fmt`, `validate` y leer el plan son la rutina mínima. El lock file va a Git.
2. `list` identifica por **posición**, `set` por **valor**, `map` por **clave**; `validation` comprueba el contenido.
3. `for_each` es la opción por defecto para colecciones con identidad; `count`, para condicionales y copias intercambiables.
4. `moved` refactoriza sin destruir infraestructura.
5. Un módulo es una **interfaz**: variables tipadas, `outputs` y `required_providers` sin `provider`; los externos, con `version` o `ref`.

Siguiente paso: laboratorio 2.